

Name: Arwind Y. Cosico

Section: BSIT-2B

PROGRAM TITLE

IT Automation Incident Ticket Manager

OBJECTIVE

The objective of this laboratory exercise is to create an Incident Ticket Management System using a linear data structure. The program allows the user to add, display, search, remove, and count incident tickets.

This activity demonstrates how a Python list can be used to store and manage multiple records in an organized way.

SCENARIO

An IT automation team receives different incident tickets from various automation bots. Each incident ticket contains an Incident ID, Bot, and Short Description.

The system is designed to help the IT team manage these incident tickets. The user can add a new ticket, display all active tickets, search for a specific ticket, remove a resolved ticket, and count the total number of active tickets.

The program starts with 10 sample incident tickets.

DATA STRUCTURED USED

The program uses a Python List, which is a type of linear data structure.

Each incident ticket is stored as a nested list containing three pieces of information:

1. Incident ID
2. Bot
3. Short Description

The program uses the following list operations:

- **append()** – adds a new incident ticket
- **for loop** – searches and displays tickets
- **remove()** – removes a resolved ticket
- **len()** – counts the active tickets

PROGRAM OPERATIONS

OPERATION	EXPLANATION
Add Incident	Gets Incident ID, Bot, and Short Description, validates them, checks for duplicate IDs, then adds the ticket using append().
Display Incidents	Traverses the incidents list and displays all active tickets.
Search Incident	Compares the entered Incident ID with each ticket until a matching ticket is found.
Remove Resolved Incident	Finds the selected Incident ID and removes the matching ticket using remove().
Count Active Incidents	Uses len(incidents) to display the current number of active tickets.

SOURCE CODE

```
incidents = [  
    ["INC1392939", "BOT-Inventory", "Failed to generate the daily report"],  
    ["INC1392940", "BOT-Email", "Failed to send the scheduled notification"],  
    ["INC1392941", "BOT-DataSync", "Encountered an error during data transfer"],  
    ["INC1392942", "BOT-Invoice", "Failed to process an invoice"],  
    ["INC1392943", "BOT-Report", "Failed to generate the weekly report"],  
    ["INC1392944", "BOT-FileTransfer", "Failed to upload the required file"],  
    ["INC1392945", "BOT-DataEntry", "Encountered an error while entering records"],  
    ["INC1392946", "BOT-Backup", "Failed to complete the scheduled backup"],  
    ["INC1392947", "BOT-Validation", "Failed to validate the submitted records"],  
    ["INC1392948", "BOT-Notification", "Failed to send the system alert"]  
]
```

```
def line():  
    print("=====")  
    print("=====")
```

```
def header(title):  
    print("\n=====")  
    print("=====")  
    print(f"|| {title:^62} ||")  
    print("=====")  
    print("=====")
```

```

def add_ticket():
    header("ADD INCIDENT TICKET")

    incident_id = input("Enter Incident ID: ").strip()

    if incident_id == "":
        print("\nERROR: Incident ID cannot be empty.")
        return

    for ticket in incidents:
        if ticket[0] == incident_id:
            print("\nERROR: Incident ID already exists.")
            return

    bot = input("Enter Bot: ").strip()

    if bot == "":
        print("\nERROR: Bot cannot be empty.")
        return

    description = input("Enter Short Description: ").strip()

    if description == "":
        print("\nERROR: Short Description cannot be empty.")
        return

    incidents.append([incident_id, bot, description])

```

```

print("\n")
print("TICKET ADDED SUCCESSFULLY")
print("\n")

```

```

def display_tickets():
    header("ACTIVE INCIDENT TICKETS")

    if len(incidents) == 0:
        print("No active incident tickets.")
        return

```

```
print("\n")
print(" | Incident ID | Bot | Short Description |")
print("\n")
```

for ticket in incidents:

```
incident_id = ticket[0][:13]
```

```
bot = ticket[1][:18]
```

```
description = ticket[2][:37]
```

```
print(f"| {incident_id:<13} | {bot:<18} | {description:<37} |")
```

```
print("Total Active Tickets: {}".format(len(incidents)))
```

```
def search_ticket():
```

```
header("SEARCH INCIDENT TICKET")
```

```
incident_id = input("Enter Incident ID: ").strip()
```

```
if incident_id == "":
```

```
print("\nERROR: Incident ID cannot be empty.")
```

return

for ticket in incidents:

```
if ticket[0] == incident id:
```

```
print("\n")
print("TICKET FOUND")
print("Incident ID : {ticket[0]:<46}")
print("Bot : {ticket[1]:<46}")
print("Description : {ticket[2]:<46}")
print()
return
```

```
print("\nERROR: Incident ID does not exist.")
```

```
def remove_ticket():
```

```
    header("REMOVE RESOLVED INCIDENT")
```

```
    incident_id = input("Enter Incident ID to remove: ").strip()
```

```
    if incident_id == "":
```

```
        print("\nERROR: Incident ID cannot be empty.")
```

```
        return
```

```
    for ticket in incidents:
```

```
        if ticket[0] == incident_id:
```

```
            incidents.remove(ticket)
```

```
    print("\n |_____|")
```

```
    print(" |      TICKET REMOVED SUCCESSFULLY      | ")
```

```
    print(" |_____|")
```

```
    return
```

```
print("\nERROR: Incident ID does not exist.")
```

```
def count_tickets():
```

```
    header("ACTIVE INCIDENT COUNT")
```

```
    print(f"\n      Total Active Tickets: {len(incidents)}")
```

```
while True:
```

```
    print("\n")
```

```
    print(" |_____|")
```

```
    print(" |      IT AUTOMATION INCIDENT MANAGER      | ")
```

```
    print(" |_____|")
```

```
    print(" | [1] Add Incident Ticket | ")
```

```
    print(" | [2] Display Active Tickets | ")
```

```
    print(" | [3] Search Incident Ticket | ")
```

```
    print(" | [4] Remove Resolved Ticket | ")
```

```

print("\n [5] Count Active Tickets\n")
print("\n [6] Exit\n")
print("\n")

choice = input("\nEnter your choice (1-6): ").strip()

if choice == "1":
    add_ticket()

elif choice == "2":
    display_tickets()

elif choice == "3":
    search_ticket()

elif choice == "4":
    remove_ticket()

elif choice == "5":
    count_tickets()

elif choice == "6":
    print("\n")
    print("\n PROGRAM TERMINATED. GOODBYE!\n")
    print("\n")
    break

else:
    print("\nERROR: Invalid choice. Please enter 1-6.")

```

PROGRAM OUTPUT

The program was tested using the 10 sample incident tickets provided in the activity. The required operations were demonstrated through the menu. The screenshots are stored in the GitHub screenshots folder.

TEST RESULT

Test	Result
Add Ticket	A new incident ticket was added successfully.
Display Tickets	The 10 sample active incident tickets were displayed.
Search Ticket	An existing Incident ID was found and its details were displayed.
Remove Ticket	INC1392943 was removed successfully.
Count Tickets	After removal during the same run, the active count is 9.

CONCLUSION

This activity demonstrated how a linear data structure can be applied to a practical IT problem. Using a Python list, incident ticket records can be stored and managed through adding, displaying, searching, removing, and counting operations. The activity also showed how these basic data structure operations can be combined to create a simple menu-driven incident management system.